

Git Methodology

Sport Coaching Tool — Project 2, COMS3011A

Version 26.08.12

1. Purpose

This document defines the git methodology followed by the team for the duration of the project: commit conventions, branching strategy, merge requirements, and versioning scheme. It applies to all repositories associated with the project and is referenced as evidence of project methodology per the course rubric.

2. Commit Conventions

Commits use scoped commit naming, in the following format:

```
<type>(<scope>): <description>
```

```
[optional body]
```

```
Assisted-by: <tool>[<model>]
```

The description is written in the imperative mood, lowercase, with no trailing period. The optional body explains the reasoning behind the change where it is not self-evident. The Assisted-by footer is included only when the commit contains AI-generated code, per the course AI policy.

2.1 Commit Types

Type	Use
feat	A new feature or capability
fix	A bug fix
docs	Documentation changes only
test	Adding or correcting tests
refactor	Code restructuring with no behavioural change
style	Formatting only, no logic change
chore	Tooling, dependencies, configuration
ci	CI/CD pipeline changes

2.2 Commit Scopes

Scopes correspond to project modules: auth, roster, events, stats, docs, ci.

2.3 Commit Frequency

Commits are made per logically complete, working unit of work, rather than on a fixed schedule. A change requiring the word "and" to describe should typically be split into separate commits.

2.4 Examples

```
feat(events): add live goal logging endpoint
fix(auth): handle expired session tokens
docs(readme): declare AI code generation usage
```

3. Branching Strategy

The team follows a trunk-based workflow with short-lived feature branches created from main.

3.1 When to Branch

- Work exceeds a trivial, single-line change
- Two or more team members may edit overlapping files concurrently

3.2 Branch Naming

```
<type>/<scope>-<short-description>
```

Examples:

feat/events-live-logging

fix/auth-session-expiry

feat/12-live-logging (when tied to an issue number)

3.3 Pull Request Description

Each pull request description must include:

- Summary of what changed and why
- Testing performed
- Link to the related issue
- AI attribution, where applicable

4. Merge Requirements

A branch may be merged into main only when all of the following conditions are met:

- Continuous integration passes (lint and automated tests)
- At least one team member approval has been given
- The branch is up to date with main
- All review comments have been resolved

Merges are performed using squash-merge, producing a single, scoped commit on main per feature.

5. Versioning Scheme

The project uses Calendar Versioning (CalVer), in the format YY.MM.DD, in place of Semantic Versioning. This reflects the milestone-driven nature of the project rather than a versioned public API, and produces a dated project history aligned with the course's milestone structure.

5.1 Release Tags

Tag	Milestone
26.08.25	Milestone 1: Sprint 1
26.09.15	Milestone 2: Sprint 2
26.09.29	Milestone 3: Sprint 3
26.10.11	Milestone 4: Submission

5.2 Tagging Procedure

```
git tag -a 26.08.25 -m "Milestone 1: Sprint 1"  
git push origin 26.08.25
```